

КАК ВЫУЧИТЬ

PYTHON

&

БЫСТРО

`print("Лучшая методика обучения")`

Дмитрий Курушкин

Как выучить Python & Быстро

Лучшая методика обучения

2-е издание

2025

Спасибо каждому читателю,
который любит Python так же, как и я

Python за 30 дней: Навигатор по изучению языка / Курушкин
Дмитрий. – 1-е изд., 2023 – 89 с., 8 илл., 3 прил.

Авторский путеводитель по обучению Python — от первых шагов до реальных проектов. Вы узнаете, как структурировать обучение, поддерживать мотивацию, избегать прокрастинации и выгорания, а также как вывести себя на уверенный уровень Junior Python-разработчика. Основано на личном опыте и практике преподавания. Подходит для новичков и тех, кто «начинал уже 5 раз, но так и не дошёл до конца».

© Курушкин Дмитрий, текст, иллюстрации 2025

Содержание

Введение	8
Об авторе	10
Почему не получается выучить язык.....	12
Неправильная мотивация.....	12
Крупная задача.....	14
Регулярность	16
Нет практики.....	17
Прокрастинация.....	19
Перегорание	20
Как изучать Python.....	22
Подготовка	22
Регулярность обучения	25
Конечная цель	25
План обучения	31
Как смотреть видеокурсы	33
Как читать книги.....	38
Как решать задачи	41

Что изучать в Python.....	44
Изучение основ	46
Базовый синтаксис.....	46
Переменные и типы данных	47
Условные операторы	47
Преобразование типов данных.....	48
Функции.....	48
Списки, кортежи, наборы и словари.....	49
Структуры данных и алгоритмы	50
Массивы и связанные списки	51
Кучи, стеки и очереди	51
Хэш-таблицы.....	52
Бинарные деревья поиска	53
Рекурсия.....	53
Алгоритмы сортировки	54
Продвинутые темы	54
Итераторы.....	55
RegEx	56
Декораторы.....	56
Lambda функции	57
Модули.....	57

ООП.....	58
Пакетные менеджеры	60
Генераторы списков	62
Парадигмы Python.....	63
Изучение фреймворков	64
Синхронные фреймворки.....	65
Асинхронные фреймворки.....	67
FastAPI	68
Тестирование.....	69
Doctest	69
Nose	70
Tornado.....	70
PyUnit / Unittest	70
Собеседование.....	71
Работа с заказчиком.....	73
Какую IDE выбрать?.....	77
Основные функции.....	77
Популярные IDE	78
На чем работаю я?	79
Полезные навыки	81
Печатание вслепую.....	81

Иллюстрации алгоритмов.....	82
Заключение.....	86
Приложения.....	87

Введение

Эта книга — не справочник по Python, и не очередной список тем, которые нужно «выучить». Это **рабочая методика**, проверенная на себе и моих учениках.

Если вы:

- только начинаете учить программирование,
- уже несколько раз начинали, но «застревали»,
- теряли мотивацию, запутывались в темах, или

не понимали, куда двигаться дальше —

то вы по адресу.

Здесь нет обещаний «закачать Python в голову за 7 дней». Вместо этого — я покажу вам **структурный маршрут**, который можно пройти в своём темпе. Вы узнаете, что действительно важно изучить, в каком порядке, как не перегореть и как двигаться вперёд даже тогда, когда кажется, что «ничего не получается».

Я постарался собрать весь свой практический опыт, инструменты и лайфхаки, которые помогут вам не просто читать и смотреть, а **действительно писать код и развиваться каждый день**.

Эта книга станет вашим **навигатором по миру Python** — с понятными развилками,

маршрутами и дорожными знаками. Всё, чтобы вы не сбились с курса и дошли до цели.

Готовы? Поехали!

Об авторе

Привет! Меня зовут Дмитрий Курушкин, и программирование стало частью моей жизни с 12 лет — когда я впервые написал HTML-страничку и увидел, как строка кода превращается в работающий сайт. Тогда это казалось магией. Сейчас — любимым ремеслом, в котором я до сих пор испытываю кайф, когда всё работает как надо.

За 14 лет я прошёл путь от школьника до руководителя собственной веб-студии. У меня за плечами:

- более **50 крупных проектов** (включая корпоративные порталы, маркетплейсы, веб-сервисы),
- **150+ сайтов**, которые я или создавал, или курировал,
- **опыт в 10+ языках программирования**, хотя **Python** — мой основной инструмент.

Я не теоретик. Всё, что вы найдёте в этой книге — это **проверенные на практике подходы**, которые помогли мне и десяткам моих учеников. Я знаю, что чувствует человек, который впервые пытается разобраться в коде. И знаю, как пройти этот путь быстро, без выгорания и вечных «затыков».

Надеюсь, вы так же, как и я, будете получать удовольствие от того, как одна строка Python может изменить целый проект — или даже вашу карьеру.

Почему не получается выучить язык

Воодушевленный идеей, непоколебимый и твердый духом сдался при изучении языка программирования через неделю.

Знакома ситуация? Если да, то не стоит отчаиваться – это проблема 90% людей, которые ежедневно заходят в программирование.

В этой главе я выделил главные причины почему не получается выучить язык.

Неправильная мотивация

Люди по разным причинам начинают заниматься программированием.

Одни рвутся за большими зарплатами в данной отрасли, удаленную работу и теплое кресло в зимнюю погоду. Другие начинают заниматься ради идеи, ради своей идеальной программы и самосовершенствования. Первые никогда не выучат язык, не смогут устроиться на высокую зарплату и тем более не смогут прогрессировать в данной деятельности. У вторых шансы велики.

Деньги – худший мотиватор на этапе обучения.

У вас сегодня денег нет, вы мотивированно занимаетесь. Завтра у Вас появляются деньги и все - мотивация утрачена. Люди так живут годами, занимаясь изучением языка по 1-2 раза в месяц. Это путь в никуда!

Периодически у меня происходит диалог с самыми разными людьми. И всегда один и тот же сценарий:

— Вот выучу Python, устроюсь на работу и буду грести бабло лопатой.

Проходит месяц. Ничего не меняется. Проходит 2 месяца, человек пишет, что все еще читает книгу и вот-вот начнет практику.

Поэтому если деньги – это единственная ваша мотивация изучить язык, то ищите другую причину для изучения. Если уверены, что другой причины нет – то не тратьте время.

Но вы можете спросить: *«Ну а как же? Работать ради идеи? А на что жить?»*. А я вам отвечу. Вы не будете работать ради идеи, вы будете обучаться ради идеи. И как только вы постигнете все знания и сможете самостоятельно писать программы с нуля – тогда уж начнете думать о деньгах. А сейчас необходимо это перенести на второй план или даже дальше.

В чем искать мотивацию? Полагаю – это история для каждого своя. Я всегда занимался и продолжаю заниматься программированием для создания именно своих приложений / сайтов / telegram ботов и так далее. Меня всегда воодушевляет идея о том, что я могу создать что-то, что может приносить мне реальную пользу (или другим людям). Плюс детский восторг от осознания того, что моя программа реально работает.

Крупная задача

Крупная задача может убить все желание заниматься ее решением. Она кажется такой большой и невыполнимой, но на самом деле это не так.

Например, я не очень люблю убираться в квартире (а есть кто-то кто любит?). Для меня всегда этот процесс очень мучителен и труден. Однако я этот процесс разбиваю на много мелких задач.

Если присмотреться более детально, то в квартире есть 2 комнаты и ванная. Получается можно прибраться сначала в одной комнате, потом в другой и третьей. В одной комнате же проще прибраться, чем в целой квартире, верно?

Но я иду дальше и делю эту подзадачу на еще более мелкие задачи. В каждой комнате мне нужно

сделать уборку мусора, пропылесосить, помыть полы, протереть пыль. В кухне добавляется еще помывка посуды и вынос мусора.

И теперь вынос мусора выглядит как ничтожная малая часть, но такая элементарная и простая. И каждая задача такая. Они просты в реализации, но выполнив их все я получаю общий результат, которым можно гордиться.

Причем если рассматривать задачу целиком, то если я не успею сделать уборку за 1 день, то я буду просто думать, что я ничего не сделал. А разбив уборку на составляющие и не успев сделать уборку, я смогу посчитать: ага... сегодня я убрал 88% квартиры. Причем смотря на этот процент, я скорее всего соберу остаток своих сил и пойду доделаю уборку до конца – вот она сила такого подхода к решению задач.

И бытовой пример, который я привел выше – это всего лишь пример. Каждую задачу можно разбивать на более мелкие задачи. И дальше, и дальше. Поэтому если вам кажется, что задача сложна – то просто поделите ее на более мелкие задачи, которые не будут Вам казаться такими ужасными и сложными.

Данный скилл очень важен для программиста. Он лежит в основе объектно-ориентированного программирования (ООП), которое и позволяет нам,

программистам, создавать по истине большие и сверхфункциональные приложения.

Обучаться мы будем по такому же принципу. Мы разделим задачи на более мелкие, так называемые темы, которые будем изучать последовательно, чтобы в конце получить лучшую версию программиста в своем лице.

Регулярность

Любая ваша задумка, идея и мечта проваливается только по одной, самой главной причине. У вас нет регулярности. Невозможно построить дом, если вы будете раз в месяц приносить по одному кирпичику.

Чтобы достичь успеха, нужно заниматься каждый день! Не обязательно тратить много времени. Но каждый ваш день должен быть связан с Python. Читайте книги, решайте задачи, продумывайте алгоритмы, смотрите тематические ролики – поглощайте информацию по языку.

Если у вас есть выбор: плотно позаниматься на выходных, скажем, 8 часов или заниматься каждый день, но по 15 минут – выбирайте второй вариант! Регулярность занятий – главная фишка для быстрого погружения в мир Python.

Казалось бы, 15 минут каждый день – это всего 1 час 45 минут в неделю. Как это может быть полезнее, чем 8 часов на выходных? Однако суть не в этом. Суть, чтобы вы привыкли уделять время языку каждый день. Вы удивитесь как прокачаетесь через 7 дней. Через 14 дней ваша прежняя жизнь будет казаться скучной. А через 21 день Python станет частью вашей жизни, и вы уже никуда не сможете от него деться. И это круто!

Нет практики

Есть такой такой тип людей. Они каждый день смотрят видео на YouTube, читают книги, смотрят стримеров, которые пишут код, но сами код не пишут.

Прокрастинаторы обманывают сами себя, убеждая, что занимаются, но на самом деле это не так.

От того, что человек пересмотрит все обучающие видео, или прочтет все книги по программированию – он не научится программировать. Посади его за компьютер он растеряется даже при написании самой простой программы.

Поэтому важно понимать, что без практики невозможно стать программистом.

Практики нужно много, качественной и бескомпромиссной. Практики нужно как минимум в 2 раза больше, чем теории. Поэтому если в день вы готовы уделять 1 час на теорию, то нужно 2 часа уделять на практику.

Практикой заниматься сложнее, чем смотреть обучающие видео или YouTube Shorts, но, когда человек пишет код самостоятельно, он задействует в том числе и мышечную память, а работа руками увеличивает активность нейронных сетей в мозге, что способствует лучшему запоминанию. Я не буду углубляться в тонкости полезности практических занятий, но важно запомнить простую истину – без практики вы не станете даже начинающим программистом.

Поэтому если человек считает, что сейчас он посмотрит тонны информации, а вот потом, когда придет время, когда ему понадобится программа, он сядет и быстро напишет код - такого не произойдет никогда. Даже если руки доберутся до компьютера, а не сложат данную задумку в долгий ящик, то при первой же проблеме в коде, мотивация что-то написать угаснет окончательно.

Прокрастинация

Прокрастинация — это когда вы откладываете дела на потом, вместо того чтобы приступить к ним сразу.

Самая отвратительная склонность, которая присуща всем людям. Разница лишь в том, можете ли Вы с ней бороться или нет.

Вы запланировали сегодня сесть и начать изучать Python в 10:00. Сейчас уже 10:02, а вы продолжаете смотреть смешные видео или играть в компьютерные игры. Вы видите время, но решаете, что начнете в 10:30... потом в 11:00... 11:30... и БАЦ! Уже вечер. Прокрастинация убила ваш сегодняшний продуктивный день? Или Вы уничтожили хорошую возможность прокачаться?

«Первая лучшая возможность, чтобы начать была год назад. Вторая лучшая возможность — прямо сейчас!».

Есть множество способов бороться с прокрастинацией. Вы можете спокойно найти их в интернете — вполне незасекреченная информация.

Но для себя я нашел ровно 1 рабочий способ, который действительно работает. Как бы странно, противоречиво или глупо он не звучал: **просто начни делать**. Не нужно торговаться с собой. Если тебе нужно делать, то просто начинай делать.

Спустя 5 минут ты втянешься и все мысли о возвращении к своим бесполезным занятиям просто исчезнут.

Перегорание

Перегорание – это обратная сторона прокрастинации. Прокрастинаторы все откладывают на потом. А есть настоящие трудоголики, которые увидев Python, начинают высасывать всю информацию до последней капли. Они готовы сидеть целыми днями за компьютером, поглощать все новую и новую информацию и изматывать себя и физически, и морально.

Как итог, перегорание. Потеря интереса к Python. Перегорание – злая штука. Если человек заполучает себе перегорание, то уже не сможет отойти за «недельку». В лучшем случае он вернется к изучению через месяц, а чаще всего уже никогда.

Установите себе четкие правила на отдых. Нельзя заниматься программированием больше 8 часов в день. Идеальные условия для обучения в день: 2 часа теории и 4 часа практики. Причем практика и теория необязательно должна идти друг за другом. Миксуйте: занимайтесь 30 минут теорией, потом 1 час практикой и так далее.

Меняйте фокус при обучении, чтобы давать себе передышку.

И самое главное! Если ощущаете, что для Вас уже много на сегодня (например, вы отзанимались 4 часа, а хотели заниматься по 8 часов), то остановитесь. Сходите покушать, попейте чаю, сделайте зарядку. Если через 15 минут вы возвращаетесь и понимаете, что почти не отдохнули – то прекращайте на сегодня, вы достигли своего лимита.

Дальнейший путь поведет Вас прямой дорогой в никуда, к перегоранию. И не получится пересилить себя. Ваш уставший мозг попросту не будет воспринимать информацию, и Вы потратите время впустую.

Важно уметь балансировать внутри себя: понимать сколько Вам лучше отдыхать, а сколько работать. И у каждого этот баланс свой. Кто-то может работать 4 часа без остановки, а кому-то каждые 45 минут нужен перерыв на 15 минут. И на самом деле это не всегда означает, что человек, который работает без остановки, будет быстрее достигать результата.

Как изучать Python

В предыдущей главе мы посмотрели на факторы, которые будут тянуть Вас назад при изучении. Искренне надеюсь, что Вы полностью прочитали советы по их преодолению и они не вызовут у вас трудности.

В этой главе я расскажу, как нужно изучать Python, чтобы достигать результата.

Подготовка

Нельзя просто сесть и начать.

Точнее можно, но тогда Вы с такой же скоростью бросите это дело, как и начали. Подготовка – это как ритуал, который закрепляет наши намерения выучить язык.

У меня есть определенное место для обучения, как дома, так и на работе. Везде это стол, удобное кресло, в котором удобно сидеть часами. Я всегда завариваю себя чай и кушаю ChocoPie. Иногда на фон включаю легкую музыку, еле слышную. Под рукой всегда находится блокнот, в который я записываю самую важную информацию.

Все это я проделываю постоянно, когда начинаю / продолжаю свое обучение. И теперь, когда я обеспечиваю себе такую обстановку, мой мозг рефлекторно переходит в режим обучения и начинает активно поглощать информацию.

Поэтому я Вам советую следующие вещи:

1. Выберите себе комфортное место для обучения. Желательно обучаться за столом, в комфортном кресле. Если у Вас ноутбук, то можете выбрать любое удобное место, главное убедитесь, что в нем Вам будет комфортно печатать. *Я пытался работать на диване, но спустя 2 часа я дико уставал печатать в таком положении, поэтому по итогу перебрался за стол и купил себе удобный стул.*
2. Обязательно обзаведитесь блокнотом и ручкой. Туда нужно будет записывать самую важную информацию и делать пометки на последующее обучение.
3. Выберите свой напиток, который вы будете попивать во время обучения. Это должно быть то, что Вам нравится: баночка колы, кофе или сок.
4. Подготовьте сладости. Обязательно сладкое, потому что сладкое полезно для мозговой активности, а у Вас ее будет очень много.
5. Придумайте что-то свое: ароматические свечи, определенная подсветка в комнате или любой

другой атрибут, который будет использоваться только во время обучения.

Вы, наверное, думаете, почему я так много времени уделяю вещам, которые, по сути, не относятся к задачам данной книги. Пролетело уже столько страниц! Однако я хочу донести до Вас такую простую мысль, которую все пропускают мимо, когда начинают заниматься программированием.

Программирование – это стиль жизнь. Это вечный процесс. Это не просто достижение целей, хотя и в этом есть доля правды. Это вечный маршрут, вечное движение. Это то, что мы будем делать ежедневно. Так почему же не сделать данный стиль жизни... стильным! Стильным под себя.

Кроме того, все эти штуки, которые мы подготовим еще до начала, создадут для нас комфортную обстановку, такую зону комфорта. На вашем пути будет бесконечное количество препятствий, мук кода, затычек и провалов. Будет много моментов, когда Вы будете на грани, когда захочется бросить все и больше никогда не начинать. И в такие моменты, я просто откидываюсь на кресло, вслушиваюсь в музыку, делаю новый глоток своего сладкого чая, закусывая чокопайкой и... просто даю себе выдохнуть в этой кайфовой обстановке, которую я создал себе заранее.

Регулярность обучения

Самая мощная привычка, которую вы можете развить в себе — это **регулярная работа с Python**. Именно она делает из «хочу» — «умею».

Здесь нет магии. Есть ритм.

Сколько времени нужно?

Если у вас есть **6 часов в день**, можно прокачаться до уровня уверенного Junior за 30–45 дней.

Это оптимальный график:

- 2 часа теории
- 4 часа практики

Но большинство людей не располагают таким временем — и это **нормально**.

Как адаптировать

Если у вас 2 часа в день:

- 40 минут теории
- 1 час 20 минут практики

Если всего 1 час:

- 20 минут теории
- 40 минут практики

Главное — **сохранять пропорции**: $\frac{1}{3}$ теории и $\frac{2}{3}$ практики. Это фундамент.

Перерывы = откат

Очень важно не выпадать из ритма. Максимум — **два дня без кода**, не больше.

Долгие перерывы отбивают навык и нарушают прогресс.

Если вы привыкли работать по графику 5/2, лучше:

- в будни заниматься по 1.5–2 часа,
- в выходные — хотя бы по 30–60 минут.

Так вы не потеряете инерцию.

Лучше каждый день по чуть-чуть, чем раз в неделю по многу.

Python должен стать вашей **ежедневной практикой**. Даже если это всего 15 минут — это 15 минут движения вперёд. А регулярное движение всегда побеждает хаотичные рывки.

Конечная цель

До того, как Вы приступите к обучению – вам нужна цель. На самом деле это **основа основ моего подхода к обучению**. Нельзя просто учиться ради того, чтобы учиться. Это самая большая оплошность нашей системы образования по моему мнению. В школе нам говорят запоминать информацию, но не говорят, где она нам пригодится. Как итог, она нам нигде и не пригождается.

Будучи на 2 курсе университета у меня, был случай, когда мы всей группой долго не могли понять одну из тем предмета. Возможно, преподаватель неправильно нам ее доносил. Поэтому наша группа решила подойти к пониманию с другой стороны. Мы спросили у преподавателя:

«Скажите нам, где это используется, где мы будем это использовать?». На что получили ответ: *«Вот вы запоминайте, а потом поймете»*. Когда мы пришли на другой предмет уже на 4 курсе, мы действительно поняли эту тему. Система образования напоминает снежный ком, который катится вниз. И с каждым новым классом или курсом действительно приходит осознание тех знаний, которые мы получили на предыдущих ступенях. Но я до сих пор считаю, что ответ преподавателя на наш вопрос был неправильным. Можно было в общих чертах нам объяснить для чего это нужно.

Но тогда я просто обучался на инженера по направлению «Мехатронные системы». Я пошел учиться, чтобы просто стать инженером. Я не особо понимал это направление. Я ходил на пары без особой цели, просто чтобы стать инженером. Не могу сказать, что мое обучение было бесполезным, однако, когда я начал изучать программирование у меня была цель – мое желание сделать сайт. И насколько данная цель увеличила мою скорость обучения – я даже не могу передать словами!

Сейчас вы купили данную книгу с четким пониманием, что хотите знать язык программирования Python. Вы примерно понимаете, что можно разработать с помощью данного языка

программирования. Поэтому пора выбрать цель данного обучения.

Придумайте проект, который вы хотите разработать. Он должен быть не сильно масштабным, но и не сильно маленьким. Не надо замахиваться на целую социальную сеть VK или мессенджер Telegram. С другой стороны, простой чат для друзей вы в состоянии сделать. Подумайте на досуге и запишите данную цель или даже список целей, которые вы хотели бы разработать. Используйте для этого тот самый блокнот, который вы подготовили.

Недавно мне пришла идея проекта, который вы можете взять себе. У меня появилась проблема при создании сайта. Сейчас на web страницах используется новый формат отображения картинок. Раньше использовался формат PNG, JPG, думаю вы знакомы с такими форматами. Сейчас используется формат AVIF, который сохраняет качество изображения, а весит очень мало. Также существует масса сайтов-конвертеров форматов (например, <https://www.iloveimg.com>). Я могу отправить картинку в формате PNG на страницу, а мне в ответ отдадут AVIF, который я могу использовать уже на сайте. Проблема заключается в том, что многие конвертеры еще не сделали функционал конвертирования в данный формат. И после долгих

поисков такого сервиса, я решил написать простое консольное приложение, которое выполняло данную задачу. Данный пример приложения прекрасен тем, что его можно реализовать как консольное приложение, web приложение, desktop приложение и можно даже упаковать в Telegram бот. Поэтому если не знаете какую цель вашего обучения выбрать, буду рад если возьмете данный проект, а потом пришлете его мне.

Зачем вообще нужно выбирать цель и как она поможет в будущем быстрее обучаться? Ответ прост. Теперь каждый урок, который вы будете проходить, каждую тему, которую будете изучать, каждый код, который будете смотреть вы будете проецировать на ваш будущий проект.

Изучая переменные, человек, который хочет разработать свою собственную игру, сразу будет понимать: *«Ага, вот переменная, она хранит данные, которыми я могу потом воспользоваться. Значит я могу эту штуку потом использовать для отображения жизни и маны моего персонажа в игре»*.

С таким подходом знания плотно закрепляются в вашей памяти и потом, даже если вы забудете сам синтаксис реализации конструкции в Python, вы будете иметь его ввиду, когда будете продумывать свои алгоритмы. Т.е. вам ничего не

помешает воспользоваться поисковиком, чтобы найти как правильно реализовать конструкцию, но не зная, что такая конструкция существует, Вы никогда не воспользуетесь ею.

На начальном этапе, при разработке первых программ, вам не нужно знать весь синтаксис и уметь воспроизводить все конструкции на память. Но очень важно знать все имеющиеся инструменты в Python, чтобы делать мощный и производительный код.

Пользуйтесь вашим блокнотом, чтобы фиксировать все конструкции, которые вы будете проходить. Используйте блочные схемы, чтобы показывать алгоритмы работы конструкций (про них мы поговорим дальше). Записывайте примеры кода. И обязательно подписывайте какие конструкции и где вы сможете применить в вашем проекте.

Я понимаю, что работать с блокнотом, да и в принципе что-то писать – это долго и муторно. Но когда Вы пишете вы задействуете мышечную память, которая способствует закреплению знаний в вашей голове. И я знаю, что есть хитрецы, которые подумали вести блокнот на планшете или компьютере, ведь так не нужно реально писать, а печатать привычнее. Так вот – это не работает. Я пробовал экспериментировать с таким подходом в

университете – он не работает. Конспектировать в планшет – не то же самое, что писать конспект от руки в блокнот. Поэтому не ленитесь и достигнете небывалых высот в скорости освоения информации!

По завершению данной главы, я надеюсь, Вы напишите в свой блокнот свою цель – программу, которую хотите разработать, будучи программистом!

План обучения

Теперь поговорим о том, как нужно обучаться. Я надеюсь, вы помните, что обучение делится на теорию и практику, которые идут бок о бок друг с другом и никак иначе.

В общем смысле вы должны следовать такому плану:

1. Подбираем курс на YouTube. Можете выбрать мой курс. Я считаю, что он хорош и максимально соответствует всему тому, что я ожидаю от хорошего курса. Либо выбирайте лектора себе по душе, кто больше нравится. В большинстве своем они рассматривают все нужные темы. Как смотреть курсы поговорим чуть дальше.
2. Обзаводимся книгой «Простой Python». Эта книга - одна из лучших. Однако ее нужно

читать именно после любого видеокурса, потому что для начинающих она покажется сложной. И страниц там много. Но вы должны побороть себя и полностью ее осилить. Как правильно читать тоже поговорим дальше.

3. Если вы серьезно отнесетесь к п.1 и п.2, полностью и должным образом пройдете эти 2 этапа, то в целом вы уже готовы творить свой код. На данном этапе вы уже можете начинать свой проект. Но параллельно я считаю будет хорошим шагом пойти на leetcode. Leetcode – это сайт, в котором собраны различные задачи по программированию. От начального уровня до сложных. Там вы найдете для себя алгоритмы, которые не написаны в книге и о которых вы не узнаете на курсах. Но опытным путем, изучая код других программистов вы придете и изучите их.
4. Берем новую книгу. Дальше вы уже должны самостоятельно выбрать книгу, которая будет соответствовать специфике вашей работы. Кто-то пойдет в нейросети, кто-то в web, кто-то еще куда-то.
5. Платный курс для повышения квалификации. Пункт необязательный, если к этому моменту вы сможете сами наметить свой путь

программиста и будете знать куда и как развиваться. Обычно это приходит само собой и человек понимает, что ему нужно. Но если все-таки решитесь погрузиться в Python глубже – приглашаю на свои курсы.

6. Практика, практика и еще раз практика. Нужно писать больше кода и решать больше поставленных задач. Когда вы окажетесь на этом пункте, Вы смело можете искать работу в интернете, либо устраиваться в IT компанию. За счет опыта будут приходить новые знания, а код будет неумолимо оттачиваться на боевых задачах.

Хочется еще раз подчеркнуть, что теория – ничто без практики. Практикуйтесь всегда и везде. Тратьте намного больше времени на практику, нежели на теорию.

И у вас все получится.

Как смотреть видеокурсы

Нельзя просто взять и начать смотреть курс.

Как вытянуть максимальную пользу от просмотра курса? За свою жизнь я пересмотрел порядка 100 различных курсов. И за это время я

выработал определенную методику просмотра, особенно, она подойдет для программирования.

Вся фишка заключается в 2-х этапном просмотре урока.

Сначала необходимо посмотреть быстро, но вдумчиво. Я обычно ставлю скорость воспроизведения на 1,5 и просматриваю урок целиком. Без практики! Это важно. Но при этом максимально вдумчиво, пытаюсь понять все, о чем говорит лектор. Если что-то непонятно, то пересматриваю отрезки еще раз. Если непонятны какие-то аббревиатуры или понятия, то отправляюсь в поиск и начинаю пополнять свой багаж понятий. Важно, чтобы после просмотра ролика Вы полностью понимали, что в нем происходит: что делает автор, почему он так делает и как вообще работает этот Python.

Помимо этого, вооружаемся блокнотиком и также выписываем все новое, что не знали до этого. Это могут быть те же понятия, алгоритмы, конструкции. Все, что Вы до этого не знали важно закрепить в своем блокноте. И опять же повторюсь, важно это писать от руки. Так лучше запоминается.

Нельзя сразу приступать к практике: т.е. включить видео – и понеслась... Так ваш просмотр будет бесполезен с точки зрения обучения. Да, формально вы протестируете конструкции,

посмотрите, как они работают, НО... во-первых, вы будете в спешке повторять за лектором и соответственно время на осмысление совсем не будет. Во-вторых, вы потратите куда больше времени, потому что постоянно будете отматывать назад.

После первого просмотра и работы с блокнотом можно приступать к практике. И здесь тоже есть важный момент. Самостоятельно все перепечатывайте. Пальцы должны привыкать к большому и муторному набору текста. А еще когда вы перепечатываете самостоятельно вы также лучше запоминаете, плюс к этому начинает работать мышечная память. И когда вы начнете писать свой код, Вы поблагодарите себя за такой труд, поверьте мне.

Помните, что при первом просмотре мы должны были выписать все непонятные и новые знания в блокнот? Так вот при перепечатывании повторяем этот ритуал, но уже внутри нашего кода. Для этого используйте комментарии в Python. Они помечаются как `#` и выглядеть все это должно примерно так:

```
# функция которая печатает в консоль  
print('Привет, Python')
```

Таким образом 1 урок мы уже обработали с 3 разных позиций: мы вдумчиво его посмотрели без практики, записали эту информацию в блокнот, продублировали информацию в коде и попрактиковали.

Именно этот тандем отложит знания, полученные в уроке на долго.

И сразу хочется оговориться по поводу комментариев. Пишите их. В обязательном порядке. Всегда. Вы должны привыкать документировать свой код, потому что, когда вы начнете писать свои программы, кода будет тонна и маленькая тележка.

Большое количество кода плохо тем, что в нем долго разбираться. Комментарии помогают быстро вспомнить все участки за считанные минуты. Причем при наличии комментариев, даже не всегда приходится читать конструкции кода, которые Вы когда-то писали. На этапе обучения очень важно привыкнуть писать комментарии.

Подытожим:

1. Смотрим урок без практики (можно включить скорость 1,5). Пытаемся понять все, о чем говорит лектор.

2. Делаем пометки в блокноте. Все новые и непонятные моменты должны оказаться в блокноте
3. Смотрим урок повторно, уже на скорости 1. Повторяем все за лектором. Делаем комментарии к коду

В таком ритме проходим весь курс. Первые темы должны пониматься и получаться легко. У начинающих программистов возникают проблемы, когда начинается объектно-ориентированное программирование (ООП). Понимание объекта и классов – сложная тема, которая входит в базовый курс. Это важные темы, их нужно пересилить и понять. На этом моменте отваливаются около 70% новичков. Но если Вы сможете пересилить все и пройти курс до конца, а главное понять его – вы будете в числе избранных.

Если сложно понимать какие-то уроки, дополнительно ищите информацию в интернете или открывайте такой же урок у другого лектора. Кто-то хорошо объясняет, кто-то плохо. Если плохо, то ищите информацию в других местах. Если объяснение хорошее, но вы не можете понять – пересматривайте видео снова и снова. В данном случае пересматривая одно и то же, действительно в какой-то момент приходит озарение и становится

сразу все понятно. Это как учить иностранные язык. В какой-то момент языковой барьер просто стирается и сразу становится легко и просто.

Как читать книги

Нельзя просто взять и читать книгу.

Немного банальностей и повторений. Советы для книги ровным счетом аналогичны, как и для видео курса.

Важно! Приступайте к книге только после того, как пройдете базовый видео курс.

Открываете книгу «Простой Python» (автор Билл Любанович). И начинаете читать. Все новые и непонятные моменты, выписывайте в блокнот. Но тут с одной оговоркой. Темы в книге будут такие же, как и в курсе. Однако книга более детально раскрывает каждую из них. Начинаете тему переменных в книге – откройте свои записи по этой теме с курса. Посмотрите какую новую информацию дает книга и пишите только новую информацию. Не нужно переписывать снова то, что уже есть в блокноте.



Рис. 1 – Книга «Простой Python»

В данной методике книга является инструментом закрепления знаний. Точно также повторяйте все примеры из книги у себя в IDE (это программа, в которой вы пишете код). Точно также везде пишите комментарии.

И самое главное: читайте! Читайте все! Не пропускайте не единой темы, даже если считаете, что данная тема простая и Вы уже все поняли.

И да, в книге очень много страниц. Вам нужно запастись терпением и силой воли. Книгу нужно осилить полностью. С проработкой, с блокнотом, с

практикой. Все это нужно сделать в обязательном порядке. Не ленитесь – делайте!

Я хочу рассказать свою историю, связанную с этой книгой. Я долго не мог ее прочитать. Постоянно откладывал, потом возвращался. И после полного прочтения я осознал, насколько сильно тормозил свою прокачку в плане программирования. Потом долго корил себя за это, да и продолжаю корить. Потому что, по сути, те 2 года, которые я читал эту книгу (да у меня заняло это реально такое время), я просто затормозился в развитии в программировании.

Когда вы прочитаете последнюю страницу данной книги, вы поймете эти прекрасные чувства: чувство прозрения, чувство понимания. У вас откроется второе дыхание для изучения все большей информации. Вы захотите выбрать свою следующую книгу! Так было у меня. Так было у моих учеников. Так будет у Вас. И после этого Вас будет уже не остановить. Вы станете машиной поглощения новой информации. Вы будете обучаться в 1000 раз быстрее, чем это было до этого. У вас появится этот детский интерес посмотреть, узнать и попробовать что-то новое, углубиться в данный процесс.

Как решать задачи

Для закрепления теории важно решать задачи, которые придумываете не только вы сами, как в случае с собственным проектом, но и когда вам дают определенную задачу для выполнения.

Есть множество сборников задач, но мне больше всего нравится leetcode (<https://leetcode.com>)

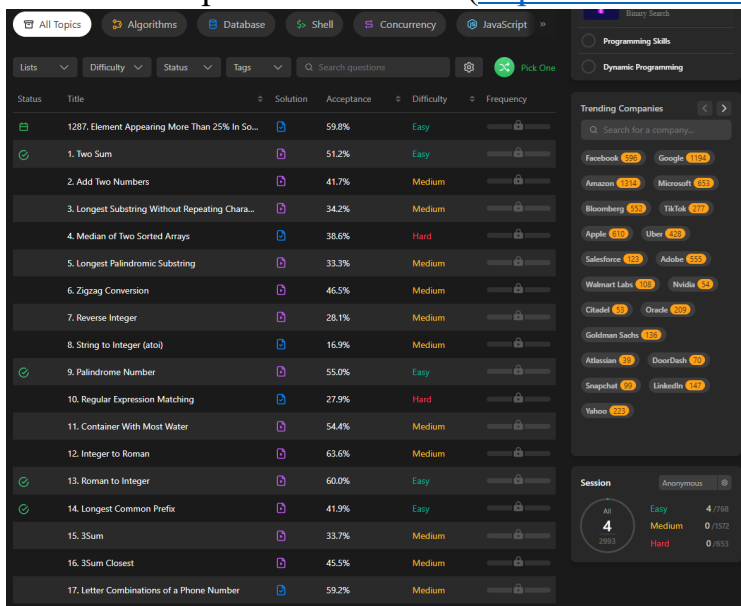


Рис. 2 – Сайт leetcode

Тут в разделе *Problems*, есть множество задач с 3 разными сложностями: легкий, средний и сложный. Что удобно, что внутри всегда пишется

полное описание к задаче, примеры выполнения и вы должны отправлять ответ. Рядом находится встроенный интерпретатор Python, поэтому код писать можно прямо на сайте.

После выполнения задачи, вы отправляете ее на проверку. Проверка осуществляется в автоматическом режиме и через 10-60 секунд вы получаете результат: правильно или неправильно вы написали программу.

По завершению выполнения задачи, вы сможете посмотреть, как выполняли данную задачу другие люди, посмотреть эффективность выполнения своего кода. А также зайти в обсуждение данной задачи, задать интересующий Вас вопрос и получить ответ от участников сообщества.

Единственный минус данного сервиса – он на английском языке. Поэтому если вы не знаток английского, то вооружитесь переводчиком или браузером (например, Yandex Browser), в котором есть встроенный переводчик. Я пользуюсь Yandex Browser и мне достаточно выделить участок английского текста, нажать правую кнопку мыши и в всплывающем окне я сразу получаю перевод. Это очень удобно. Если вы ярый противник Yandex, то можете найти расширение для Google Chrome.

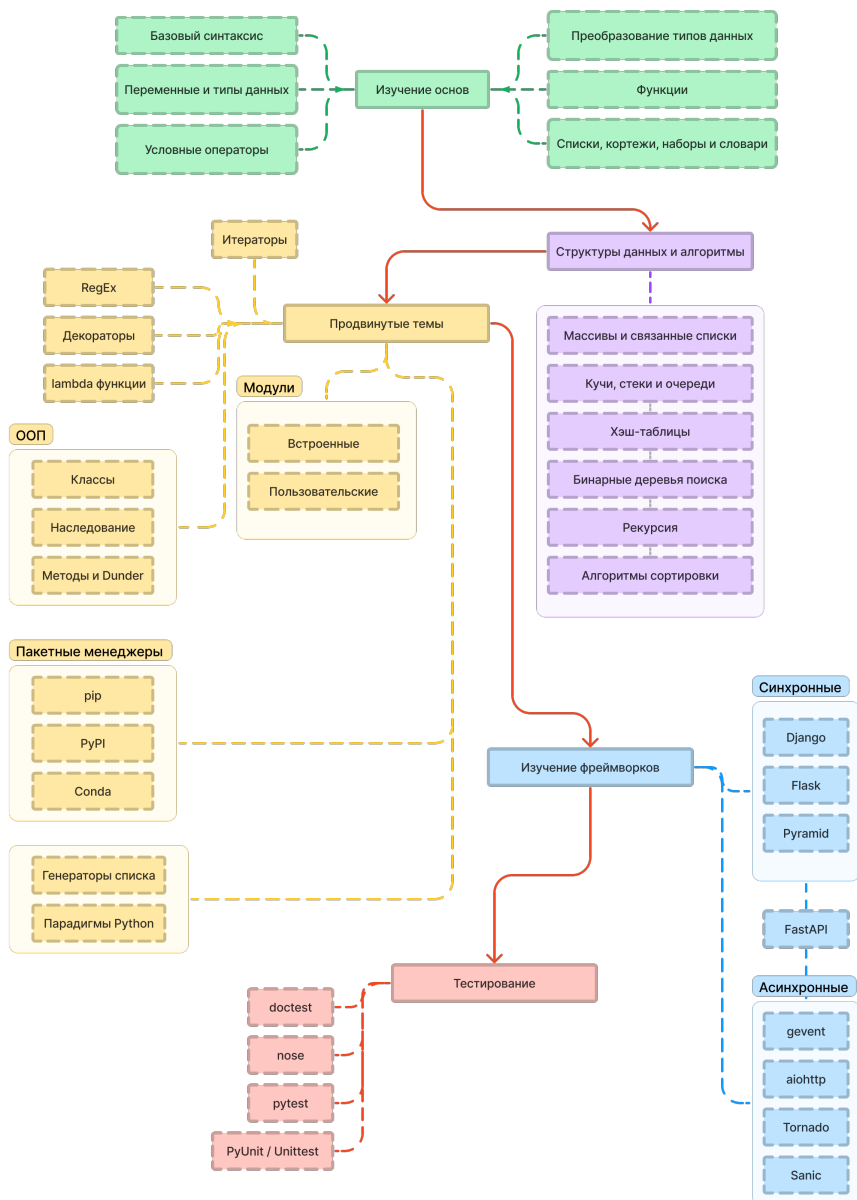
Задач в данном сервисе очень много (порядка 3000). И решать их в последовательности от легкого к сложному не имеет никакого смысла. Решайте в день по 3 задачи: одну легкого уровня, одну среднего и одну сложного. Если не получается решать сложную задачу, то заменяйте ее задачей среднего уровня. В день вы должны решать по 3 задачи.

Что изучать в Python

Мы уже познакомились с методикой изучения языка программирования Python, и вы уже знаете, КАК обучаться. Теперь я Вам расскажу, ЧТО нужно изучать.

Для этого я подготовил для вас так называемую дорожную карту (roadmap) для изучения Python.

Дорожная карта уже ждет вас на следующей странице. Далее данный раздел будет следовать дорожной карте. Мы будем двигаться по маршруту, и я буду обозревать темы, которые должен знать каждый разработчик.



Изучение основ

Python - это высокоуровневый, интерпретируемый язык программирования общего назначения. Философия его дизайна подчеркивает удобочитаемость кода с использованием значительных отступов. Python динамически типизируется, что позволяет не думать про типы данных.

Данный шаг вы пройдете в ходе любого видеокурса. Сюда входят следующие обязательные темы: базовый синтаксис, переменные и типы данных, условные операторы, преобразование типов данных, функции, а также списки, кортежи, наборы и словари.

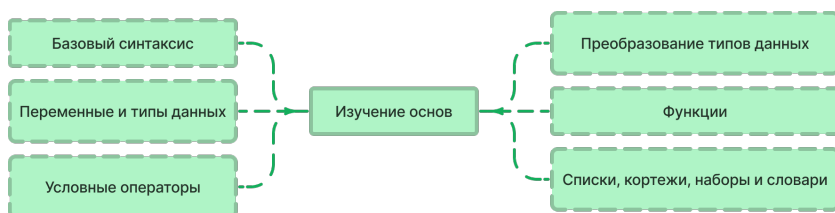


Рис. 3 – Раздел «Изучение основ»

Базовый синтаксис

В данной теме вы настроите среду для python и изучите основы.

Переменные и типы данных

Переменные используются для хранения информации, на которую можно ссылаться и которой можно манипулировать в компьютерной программе. Они также предоставляют способ присвоения данным описательного названия, чтобы наши программы были более понятны читателю и нам самим. Полезно думать о переменных как о контейнерах, содержащих информацию. Их единственная цель - маркировать и сохранять данные в памяти. Затем эти данные можно использовать во всей вашей программе.

Изучите данную тему на видеокурсе и в учебнике «Простой Python» 2016, с. 43.

Условные операторы

Условные операторы в Python выполняют различные действия в зависимости от того, принимает ли конкретное условие значение true или false. Условные операторы обрабатываются операторами IF-ELIF-ELSE и операторами MATCH-CASE в Python.

Изучите данную тему на видеокурсе и в учебнике «Простой Python» 2016, с. 102.

Преобразование типов данных

Процесс преобразования значения одного типа данных (integer, string, float и т.д.) в другой тип данных называется преобразованием типов. В Python есть два типа преобразования типов: неявное и явное.

Изучите данную тему на видеокурсе и в учебнике «Простой Python» 2016, с. 53.

Функции

В программировании функция — это повторно используемый блок кода, который выполняет определенную функциональность при ее вызове. Функции являются неотъемлемой частью каждого языка программирования, поскольку они помогают сделать ваш код более модульным и пригодным для повторного использования.

В Python вы определяете функцию с помощью ключевого слова `def`, затем записываете идентификатор функции (имя), за которым следуют круглые скобки и двоеточие.

Изучите данную тему на видеокурсе и в учебнике «Простой Python» 2016, с. 118.

Списки, кортежи, наборы и словари

Списки: похожи на массивы динамического размера, объявленные на других языках (vector в C++ и ArrayList в Java). Списки не обязательно всегда должны быть однородными, что делает их самым мощным инструментом в Python.

Кортеж: Кортеж — это набор объектов Python, разделенных запятыми. В некотором смысле кортеж похож на список с точки зрения индексации, вложенных объектов и повторения, но кортеж неизменяем, в отличие от изменяемых списков.

Set: Набор — это неупорядоченный тип данных коллекции, который является повторяемым, изменяемым и не имеет повторяющихся элементов. Класс set в Python представляет математическое понятие множества.

Словарь: В python словарь - это упорядоченный набор значений данных, используемый для хранения значений данных наподобие карты, которая, в отличие от других типов данных, которые содержат только одно значение в качестве элемента. Словарь содержит пару *ключ:значение*. Ключ-значение предоставляется в словаре, чтобы сделать его более оптимизированным.

Изучите данную тему на видеокурсе и в учебнике «Простой Python» 2016, с. 70.

Структуры данных и алгоритмы

Структура данных — это именованное местоположение, которое может использоваться для хранения и организации данных. Алгоритм — это набор шагов для решения конкретной задачи. Изучение структур данных и алгоритмов позволяет нам писать эффективные и оптимизированные компьютерные программы.

Чтобы разобраться в данной теме, я предлагаю вам ознакомиться с книгой Дональда Р. Шихи «Структуры данных в Python: начальный курс».



Рис. 4 – Раздел «Структуры данных»

Массивы и связанные списки

Массивы хранят элементы в смежных ячейках памяти, что приводит к легко вычисляемым адресам для сохраненных элементов, и это обеспечивает более быстрый доступ к элементу с определенным индексом. Связанные списки имеют менее жесткую структуру хранения, и элементы обычно хранятся не в смежных местоположениях, следовательно, их необходимо хранить с дополнительными тегам, дающими ссылку на следующий элемент. Это различие в схеме хранения данных определяет, какая структура данных будет более подходящей для данной ситуации.

Изучите данную тему в книге «Структуры данных в Python: начальный курс» 2022, с. 59.

Кучи, стеки и очереди

Стеки: Операции выполняются LIFO (последний вход, первый выход), что означает, что последний добавленный элемент будет удален первым. Стек может быть реализован с использованием массива или связанного списка. Если в стеке заканчивается память, это называется переполнением стека.

Очередь: Операции выполняются по FIFO (первый вход, первый выход), что означает, что первый добавленный элемент будет первым удаленным. Очередь может быть реализована с использованием массива.

Куча: Древовидная структура данных, в которой значение родительского узла упорядочено определенным образом по отношению к значению его дочерних узлов. Куча может быть либо минимальной кучей (значение родительского узла меньше или равно значению его дочерних узлов), либо максимальной кучей (значение родительского узла больше или равно значению его дочерних узлов).

Изучите данную тему в книге «Структуры данных в Python: начальный курс» 2022, с. 53.

Хэш-таблицы

Хэш-таблица, карта, HashMap — все это названия одной и той же структуры данных. Это структура данных, которая реализует определенный абстрактный тип данных, структуру, которая может сопоставлять ключи значениям.

Изучите данную тему в книге «Структуры данных в Python: начальный курс» 2022, с. 109.

Бинарные деревья поиска

Двоичное дерево поиска, также называемое упорядоченным или отсортированным двоичным деревом, представляет собой корневую структуру данных двоичного дерева, в которой ключ каждого внутреннего узла больше, чем все ключи в левом поддереве соответствующего узла, и меньше, чем ключи в его правом поддереве.

Изучите данную тему в книге «Структуры данных в Python: начальный курс» 2022, с. 120-144.

Рекурсия

Рекурсия — это метод решения вычислительной задачи, при котором решение зависит от решений меньших экземпляров одной и той же задачи. Рекурсия решает такие рекурсивные задачи с помощью функций, которые вызывают сами себя из своего собственного кода.

Изучите данную тему в книге «Структуры данных в Python: начальный курс» 2022, с. 74.

Алгоритмы сортировки

Сортировка относится к упорядочиванию данных в определенном формате. Алгоритм сортировки определяет способ упорядочивания данных в определенном порядке. Наиболее распространенные порядки - в числовом или лексикографическом порядке.

Важность сортировки заключается в том факте, что поиск данных может быть оптимизирован до очень высокого уровня, если данные хранятся в отсортированном виде.

Изучите данную тему в книге «Структуры данных в Python: начальный курс» 2022, с. 89.

Продвинутые темы

Теперь, когда вы рассмотрели основы Python, давайте перейдем к некоторым продвинутым темам. В этом разделе вы узнаете о таких вещах, как ООП, лямбда-выражения, декораторы, итераторы, модули и многое другое.

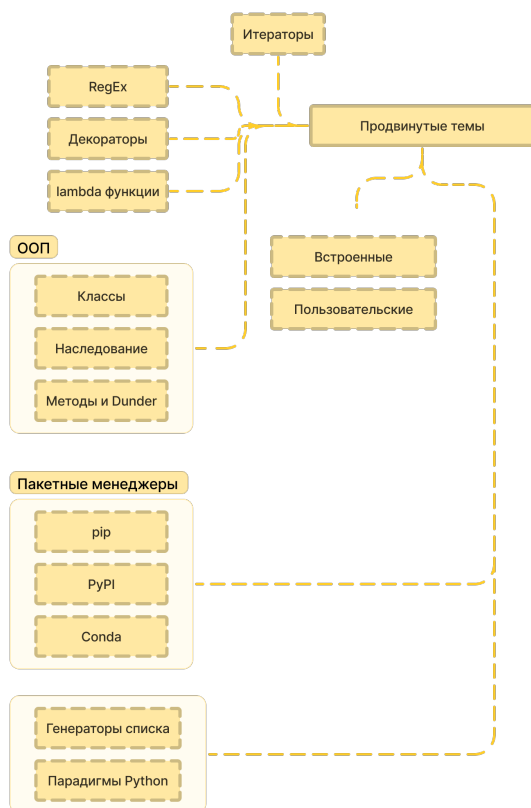


Рис. 5 – Раздел «Продвинутые темы»

Итераторы

Итератор — это объект, содержащий счетное число значений. Итератор — это объект, по которому можно выполнять итерации, что означает,

что вы можете перемещаться по всем значениям. Технически, в Python итератор - это объект, который реализует протокол итератора, состоящий из методов `iter()` и `next()` .

RegEx

Регулярное выражение — это последовательность символов, которая задает шаблон поиска в тексте. Обычно такие шаблоны используются алгоритмами поиска строк для операций “найти” или “найти и заменить” над строками или для проверки ввода.

Изучите данную тему в учебнике «Простой Python» 2016, с. 192.

Декораторы

Декоратор — это шаблон проектирования в Python, который позволяет пользователю добавлять новые функциональные возможности к существующему объекту без изменения его структуры. Декораторы обычно вызываются перед определением функции, которую вы хотите украсить.

Изучите данную тему в учебнике «Простой Python» 2016, с. 132.

Lambda функции

Лямбда-функции Python являются анонимными, что означает, что функция не имеет имени. Как мы уже знаем, ключевое слово `def` используется для определения обычной функции в Python. Аналогично, ключевое слово `lambda` используется для определения анонимной функции в Python.

Изучите данную тему в учебнике «Простой Python» 2016, с. 130.

Модули

Модули ссылаются на файл, содержащий инструкции и определения Python. Файл, содержащий код Python, например: `example.py`, называется модулем, и его имя модуля будет `example`. Мы используем модули для разбивки больших программ на небольшие управляемые и организованные файлы. Кроме того, модули обеспечивают возможность повторного использования кода.

Изучите данную тему в учебнике «Простой Python» 2016, с. 143.

Встроенные модули

Интерпретатор Python имеет ряд встроенных функций. Они всегда доступны для использования в каждом сеансе интерпретатора. Многие из них обсуждались ранее. Например, `print()` и `input()` для ввода-вывода, функции преобразования чисел (`int()`, `float()`, `complex()`), преобразования типов данных (`list()`, `tuple()`, `set()`) и т.д.

Пользовательские модули

Модули ссылаются на файл, содержащий инструкции и определения Python. Файл, содержащий код Python, например: `example.py`, называется модулем, и его имя модуля будет `example`. Мы используем модули для разбивки больших программ на небольшие управляемые и организованные файлы. Кроме того, модули обеспечивают возможность повторного использования кода.

ООП

В Python объектно-ориентированное программирование (ООП) — это парадигма программирования, которая использует объекты и классы в программировании. Она направлена на реализацию в программировании объектов реального мира, таких как наследование, полиморфизмы, инкапсуляция и т.д. Основная

концепция ООП заключается в том, чтобы связать данные и функции, которые с ними работают, как единое целое, чтобы никакая другая часть кода не могла получить доступ к этим данным.

Классы

Класс — это определяемая пользователем схема или прототип, на основе которого создаются объекты. Классы предоставляют средства объединения данных и функциональных возможностей. Создание нового класса создает объект нового типа, позволяя создавать новые экземпляры этого типа. К каждому экземпляру класса могут быть прикреплены атрибуты для поддержания его состояния. Экземпляры класса также могут иметь методы (определенные их классом) для изменения их состояния.

Изучите данную тему в учебнике «Простой Python» 2016, с. 157.

Наследование

Наследование позволяет нам определить класс, который наследует все методы и свойства от другого класса.

Изучите данную тему в учебнике «Простой Python» 2016, с. 160.

Методы и Dunder

Метод в python чем-то похож на функцию, за исключением того, что он связан с

объектом/классами. Методы в python очень похожи на функции, за исключением двух основных отличий.

- Метод неявно используется для объекта, для которого он вызывается.
- Метод доступен для данных, содержащихся в классе.

Методы Dunder или magic в Python — это методы, имеющие два префикса и суффикс подчеркивания в имени метода. Dunder здесь означает “Двойное подчеркивание (Underscores)”. Они обычно используются для перегрузки операторов. Несколько примеров магических методов: `init`, `add`, `len`, `repr` и т.д.

Изучите данную тему в учебнике «Простой Python» 2016, с. 162.

Пакетные менеджеры

Менеджеры пакетов позволяют вам управлять зависимостями (внешним кодом, написанным вами или кем-то другим), которые необходимы вашему проекту для корректной работы.

PyPI и Pip - наиболее распространенные кандидаты, но есть и некоторые другие доступные варианты:

- Poetry: управляет зависимостями с помощью изоляции
- REX: Развертывание приложений на основе изоляции, поэтому вам не нужно влиять на системные или пользовательские библиотеки REX. Это позволяет вам опробовать отдельные инструменты python CLI, не затрагивая другие зависимости.

PyPI

PyPI, обычно произносимое как pie-pee-pee, представляет собой хранилище, содержащее несколько сотен тысяч пакетов. Они варьируются от тривиальных реализаций Hello, World до продвинутых библиотек глубокого обучения.

Pip

Стандартным менеджером пакетов для Python является pip. Он позволяет устанавливать пакеты, которые не являются частью стандартной библиотеки Python, и управлять ими.

Conda

Conda - это система управления пакетами и средой с открытым исходным кодом, которая работает в Windows, macOS и Linux. Conda быстро устанавливает, запускает и обновляет пакеты и их зависимости. Conda легко создает, сохраняет, загружает и переключается между средами на вашем локальном компьютере. Он был создан для

программ на Python, но может упаковывать и распространять программное обеспечение для любого языка.

Conda как менеджер пакетов помогает вам находить и устанавливать пакеты. Если вам нужен пакет, для которого требуется другая версия Python, вам не нужно переключаться на другой менеджер среды, потому что conda также является менеджером среды. С помощью всего лишь нескольких команд вы можете настроить совершенно отдельную среду для запуска этой другой версии Python, продолжая запускать вашу обычную версию Python в вашей обычной среде.

Генераторы списков

Генератор списков — это краткий способ создания списка с использованием одной строки кода на Python. Они являются мощным инструментом для создания списков и манипулирования ими, и их можно использовать для упрощения и сокращения кода.

Изучите данную тему в учебнике «Простой Python» 2016, с. 131.

Парадигмы Python

Python - многопарадигмальный язык программирования, что означает, что он поддерживает несколько парадигм программирования. Некоторые из основных парадигм, поддерживаемых Python, следующие:

Императивное программирование: Эта парадигма фокусируется на том, чтобы шаг за шагом указывать компьютеру, что делать. Python поддерживает императивное программирование с такими функциями, как переменные, циклы и управляющие структуры.

Объектно-ориентированное программирование (ООП): Эта парадигма основана на идее объектов и их взаимодействий. Python поддерживает ООП с такими функциями, как классы, наследование и полиморфизм.

Функциональное программирование: Эта парадигма основана на идее функций как граждан первого класса и подчеркивает использование чистых функций и неизменяемых данных. Python поддерживает функциональное программирование с такими функциями, как функции высшего порядка, лямбда-выражения и генераторы.

Аспектно-ориентированное программирование: Эта парадигма основана на идее

отделения сквозных задач от основной функциональности программы. В Python нет встроенной поддержки аспектно-ориентированного программирования, но это может быть достигнуто с помощью библиотек или языковых расширений.

Поддержка Python нескольких парадигм делает его универсальным и гибким языком и позволяет разработчикам выбирать парадигму, которая наилучшим образом соответствует их потребностям.

Изучение фреймворков

Фреймворки автоматизируют общую реализацию общих решений, что дает пользователям возможность сосредоточиться на логике приложения, а не на основных рутинных процессах.

Фреймворки облегчают жизнь веб-разработчикам, предоставляя им структуру для разработки приложений. Они предоставляют общие шаблоны в веб-приложении, которые являются быстрыми, надежными и легко обслуживаемыми.

Фреймворки можно разделить на 2 категории: синхронные и асинхронные

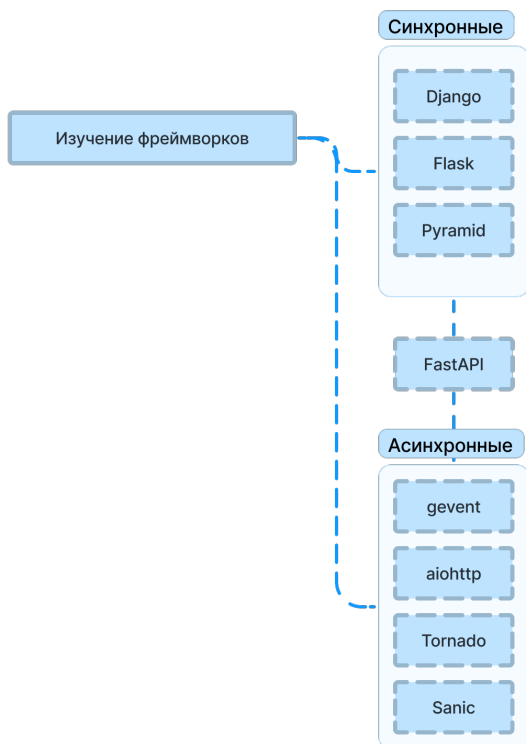


Рис. 6 – Раздел «Фреймворки»

Синхронные фреймворки

Синхронные фреймворки в python обрабатывают поток данных синхронным образом. При синхронном запросе вы отправляете запрос и прекращаете выполнение своей программы до тех пор, пока не получите ответ от HTTP-сервера (или сообщение об ошибке, если сервер недоступен, или

тайм-аут, если соединение занимает слишком много времени для ответа). Интерпретатор блокируется до тех пор, пока запрос не будет завершен (пока вы не получите окончательный ответ о том, что произошло с запросом: все прошло хорошо? была ли ошибка? тайм-аут?).

К синхронные фреймворкам относятся следующие: Django, Flask, Pyramid.

1. Django

Django - это бесплатный веб-фреймворк на основе Python с открытым исходным кодом, который следует архитектурному шаблону model-template-views. Он поддерживается Django Software Foundation, независимой организацией, созданной в США как некоммерческая организация 501.

2. Flask

Flask - это микро-веб-фреймворк, написанный на Python. Он классифицируется как микро-фреймворк, поскольку для него не требуются специальные инструменты или библиотеки. В нем нет уровня абстракции базы данных, проверки формы или любых других компонентов, где ранее существовавшие сторонние библиотеки предоставляют общие функции.

3. Pyramid

Pyramid - это общая платформа разработки веб-приложений с открытым исходным кодом,

построенная на python. Она позволяет разработчику python с легкостью создавать веб-приложения. Pyramid поддерживается корпоративной системой управления знаниями KARL (проект Джорджа Сороса).

Асинхронные фреймворки

Асинхронное программирование — это тип параллельного программирования, в котором единице работы разрешено выполняться отдельно от основного потока приложения. Когда работа завершена, она уведомляет основной поток о завершении или сбое рабочего потока. Этот стиль в основном связан с асинхронным выполнением задач. Python имеет несколько асинхронных фреймворков, которые используются для реализации асинхронного программирования.

К асинхронным относятся следующие фреймворки: `gevent`, `aiohttp`, `Tornado`, `Sanic`.

1. Gevent

`gevent` - это библиотека Python, предоставляющая высокоуровневый интерфейс для цикла обработки событий. Она основана на неблокирующем вводе-выводе (`libevent/libev`) и облегченных `greenlets`. Неблокирующий ввод-вывод означает, что запросы, ожидающие сетевого ввода-

вывода, не будут блокировать другие запросы; гринлеты означают, что мы можем продолжать писать код в синхронном стиле.

2. Aiohttp

aiohttp - это библиотека Python 3.5+, которая обеспечивает простую и мощную асинхронную реализацию HTTP-клиента и сервера.

3. Tornado

Tornado - это масштабируемый, неблокирующий веб-сервер и платформа веб-приложений, написанные на Python. Он был разработан для использования FriendFeed; компания была приобретена Facebook в 2009 году, и вскоре после этого Tornado был доступен с открытым исходным кодом.

4. Sanic

Sanic — это веб-сервер и веб-фреймворк на Python 3.7+, написанные для быстрой работы. Он позволяет использовать синтаксис `async/await`, добавленный в Python 3.5, что делает ваш код неблокирующим и быстрым.

FastAPI

FastAPI — это веб-фреймворк для разработки RESTful API на Python. FastAPI основан на Pydantic и подсказках типа для проверки, сериализации и

десериализации данных и автоматической генерации документов OpenAPI.

Тестирование

Ключом к созданию программного обеспечения, отвечающего требованиям без дефектов, является тестирование. Тестирование программного обеспечения помогает разработчикам понять, что они создают правильное программное обеспечение. Когда тесты выполняются как часть процесса разработки (часто с использованием инструментов непрерывной интеграции), они укрепляют уверенность и предотвращают регрессии в коде.

Doctest

Стандартная библиотека Python поставляется с модулем `test framework` под названием `doctest`. Модуль `doctest` программно выполняет поиск в коде Python фрагментов текста в комментариях, которые выглядят как интерактивные сеансы Python. Затем модуль выполняет эти сеансы, чтобы подтвердить, что код, на который ссылается `doctest`, выполняется должным образом.

Nose

Nose - это еще одна платформа тестирования с открытым исходным кодом, которая расширяет unittest, обеспечивая более гибкую платформу тестирования.

Tornado

Tornado - это масштабируемый, неблокирующий веб-сервер и платформа веб-приложений, написанные на Python. Он был разработан для использования FriendFeed; компания была приобретена Facebook в 2009 году, и вскоре после этого Tornado был доступен с открытым исходным кодом.

PyUnit / unittest

PyUnit - это простой способ создания программ модульного тестирования и UnitTests с помощью Python. (Обратите внимание, что в docs.python.org используется имя “unittest”, которое также является именем модуля.)

Собеседование

Перед тем, как начать работать и получать за это реальный кэш, осталось пройти последнее испытание – собеседование. Здесь хочу перечислить советы, которые я даю свои ученикам.

Во-первых, не стоит заикливаться на одной компании. При просмотре вакансий всегда выбирайте целый ряд компаний, которые подходят по вашим умениям, удовлетворяют вашим потребностям в зарплате и... проходите собеседования во всех них. Помните, что Вас никто не гонит с принятием решения. А пройдя несколько собеседований у вас набор данных для сравнения.

Мой ученик всегда мечтал попасть в Яндекс, ну, потому что это же Яндекс – одна из топовых IT компаний в России. Однако я посоветовал не торопиться, и попробовать пройти собеседования у других компаний. Как итог, он прошел в Яндекс, но решил отказаться, потому что другая компания предложила намного лучше условия и оплату труда. И я не в коем случае не хочу задеть нежные чувства Яндекса, или сказать, что не идите туда. Но стоит учитывать, что еженедельно ситуация на рынке меняется. Предложения по вакансиям меняются тоже.

Во-вторых, смотрите как с вами общаются на собеседовании. Диалог должен быть приятным и понятным. Напряжения не должно витать в воздухе. Понятное дело, что от волнения вы никуда не уйдете, тем более, если это ваше первое собеседование. Однако на 3-4 собеседование вас уже не будет это заботить. Собеседование обычно проводит ваш будущий руководитель, поэтому сразу обратите внимание на общение с ним. Если общение не задается, то работать с данным руководителем вам будет сложно, даже если Вы пройдете.

Работа с заказчиком

В целом, можно и не идти работать на дядю. А гордо и с поднятой головой пойти работать на себя. В этом есть явные преимущества: не надо работать по графику, не нужно никому подчиняться, не надо даже ездить на работу. Работаешь спокойно из дома, никуда не встаешь рано утром – красота.

Однако тут есть подводные камни.

Во-первых, наличие заказов. Оно не всегда есть или есть всегда, но не в должном количестве. Например, условно вам нужно делать каждый месяц 3 заказа по 40т.р. чтобы выходить в свои комфортные 120т.р. за месяц. Но что, если заказ будет всего 1? А если их совсем не будет? С другой стороны заказов может быть намного больше и не редкость, что даже джуны на фрилансе зарабатывают больше, чем сеньоры в IT компании. Главное найти свой поток клиентов. Но в любом случае надо понимать, что работа на фрилансе – это качели. То заказы есть, то их нет. К этому важно быть готовым.

Во-вторых, надо вести прямой диалог с заказчиком. Работая на IT компанию, вы либо работаете над продуктом компании, как например трудиться в Яндекс и разрабатывать

Яндекс.Музыку, либо работаете над заказом, где вам приносят готовое ТЗ. В обоих случаях вы Заказчика не видите, с ним общается ваш руководитель или руководитель руководителя. А руководитель, уже отфильтровав, всю полученную информацию, передает только важную информацию для вашей работы. Т.е. есть всегда один постоянный человек, который дает вам задачи. Это ваш руководитель. И это плюс, потому что с руководителем спустя время вы притираетесь и легко понимаете, что от Вас хотят. С заказчиком все намного сложнее. Заказчики сами по себе натуры сложные – они не понимают, чего хотят, но им надо и они, разумеется, не имея опыта в программировании, все равно знают лучше, как вам делать работу.

Поэтому общение с Заказчиком – это отдельный треш для фрилансера.

В-третьих, нет поддержки. Если вы начинаете работать на себя – вы должны быть полноценным специалистом, который может решать любые задачи. У вас сразу в голове должно образовываться много путей решения проблемы заказчика, чтобы в случае провала первого пути решения, у вас в запасе были еще варианты. Потому что в случае затыка при программировании – некому будет помочь. Начинаете долго решать проблему – заказчик

начинает нервничать. Плюс к этому все упирается в сроки выполнения заказа. В случае с IT компанией там есть кто сможет вам подсобить и помочь – в данном случае такого варианта нет.

В-четвертых, ненормированный рабочий график и неправильное распределение времени. Я с этим часто сталкивался в начале пути. Заказы идут, люди хотят заплатить денег. Что надо делать? Ну конечно же брать и делать! Так я думал. Ведь много сделаешь – много заработаешь. Однако тут кроется проблема с выгоранием и нервозностью. Заказов много, сроки поставлены. И сначала все ОК. Но потом вылезает затык по одному заказу, потом по второму. Сроки горят, уровень стресса повышается. Рабочий день приходится повышать, усталость растет. И после такого «забега» приходится брать недельку-две перерыва, что в общем смысле не приводит к высокому доходу. Ну а что? Ты поработал плотно 2 недели, 2 недели отдохнул. За месяц получилось также, как и за месяц равномерной работы. Причем потом еще сложно вернуться в привычный рабочий темп. Т.е. при такой работе получается, что много времени теряется, а это значит денег зарабатывается меньше.

В IT компаниях распределением рабочего времени занимается отдельный человек. Он ставит реальные комфортные сроки выполнения, что

исключается выгорание и делает работу продуктивной. А программист об этом даже не задумывается.

Получается нужно реально оценивать свои возможности. Особенно на начальном этапе работы на себя. Лучше работать равномерно, но постоянно. Рано или поздно скорость выполнения заказов будет увеличиваться и это не будет приводить к выгоранию. Наращивать количество заказов нужно постепенно.

В целом, мой совет тут простой – нужно идти сначала в IT компанию. Там можно набраться нужного опыта, понять, как работают люди, влиться в рабочий процесс. И только потом переходить на фриланс. Так вы сможете оказывать качественные услуги с быстрым сроком выполнения.

Какую IDE выбрать?

IDE (Integrated Development Environment) — это программное обеспечение, предоставляющее средства разработки приложений в единой среде. IDE интегрирует в себе различные инструменты и функции, упрощающие процесс создания программного обеспечения, включая редакторы кода, компиляторы, отладчики, автоматические системы сборки и другие инструменты для разработки.

Основные функции

1. Редактор кода: позволяет программистам писать, редактировать и форматировать код.

2. Компилятор и интерпретатор: предоставляет инструменты для компиляции и выполнения кода.

3. Отладчик: помогает выявлять и исправлять ошибки в коде, предоставляя инструменты для шагового выполнения кода, просмотра значений переменных и других отладочных операций.

4. Система сборки: позволяет автоматизировать процесс компиляции и сборки программы.

5. Управление проектами: предоставляет средства для создания, редактирования и управления проектами и файлами.

6. Интеграция с системами контроля версий: Многие современные IDE предоставляют интеграцию с популярными системами контроля версий, такими как Git.

Популярные IDE

1. Visual Studio от Microsoft для разработки приложений на .NET и других языках программирования.

2. Eclipse — мощное и гибкое средство для разработки на множестве языков программирования.

3. IntelliJ IDEA — популярная IDE для разработки на Java и других языках.

4. PyCharm — IDE для разработки на Python.

5. Xcode — IDE для разработки приложений под macOS и iOS от Apple.

Использование IDE значительно упрощает и ускоряет процесс разработки программного обеспечения, предоставляя программистам интегрированную среду с необходимыми инструментами и функциями.

На чем работаю я?

Долгое время я работал на PyCharm. Отличная IDE с кучей возможностей из коробки. Бесплатная версия функциональная и полностью закрывает все ваши потребности во время обучения и долго будет закрывать во время работы. Скачать можно тут (<https://www.jetbrains.com/pycharm/>).

Однако в один момент мне понадобилась работа с удаленным сервером. И данный вариант в бесплатной версии не предусмотрен. Только платная версия IDE позволяет делать такую штуку. Покупать, разумеется, не хотелось.

Поэтому я перешел на Visual Studio (<https://code.visualstudio.com/>). Эта IDE полностью бесплатная, имеет куча дополнений в виде плагинов. Долго адаптировался, потому что после Pycharm, где все было сразу, тут ты, по сути, скачиваешь пустую оболочку, в которую надо нагрузить персонализированные плагины.

Однако вскоре понимаешь, что такая персонализация – даже лучше! Плюсом данная IDE поддерживает все языки программирования, в отличии от PyCharm.

В целом советую сразу открывать мир программирования с Visual Studio. Pycharm хоть и хорош и греет мне душу до сих пор, однако

адаптироваться в VS для меня стало сложно после него. А если планируете расти в программировании, то вам явно нужен инструмент, который не ограничен в функционале.

Полезные навыки

Заключительным аккордом данной книги, я хочу рассказать про полезные, но необязательные навыки, которые увеличат вашу производительность. Эти навыки не являются обязательными и про них не будут у вас спрашивать на собеседовании. Однако имея их в арсенале вы ускорите свое развитие в разы.

Печатание вслепую

Один из самых крутых навыков, которым я горжусь, хоть и не владею им в идеале — это печатание вслепую. Думаю, не стоит объяснять, что скорость печатания напрямую влияет на то, как быстро вы напишите свой код. Быстрее напишите — больше сделаете за 1 условный промежуток времени. Продуктивность возрастает.

Основным плюсом умения является тот факт, что вы постоянно можете смотреть на экран и видеть, что у вас происходит в вашем коде. Когда структура кода большая и сложная — это архиважно, потому смена фокуса может улетучить ваши мысли и вам придется снова вспоминать, какое решение вы

придумали. Да и когда вы просто меняете фокус – устаете быстрее.

Когда я проходил повышение квалификации по JavaScript, мой лектор очень много времени тратил на перепечатывание кода – он просто забывал переключиться на английскую раскладку.

Поэтому я настоятельно рекомендую обучиться данному навыку. Есть масса сайтов-тренажёров по слепой печати. Вот, например, <https://stamina-online.com/ru>.

Иллюстрации алгоритмов

Если мы говорим про полноценные сервисы, то важно изначально продумать правильную архитектуру вашего приложения. Неправильная структура может породить множество проблем в будущем. А чем больше уже написано, тем сложнее и мучительнее будет все это переписывать, если программист в какой-то момент понимает, что структура то неправильная! И когда это осознание приходит перед кодером есть 2 выбора: переписать вообще все, но правильно (а это уже тысячи строк кода), либо писать дальше и использовать костыли программы. Большинство программистов выбирают второй вариант и на выходе получают

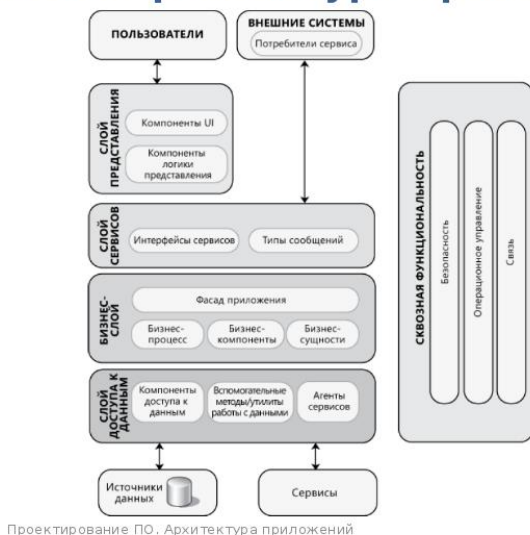
непроизводительное и несмаштабируемое приложение.

Решение данной проблемы на самом деле простое. Перед началом программирования нужно нарисовать структуру вашего приложения. Просто взять листочек и порисовать. Прикинуть разные варианты и выбрать лучший из них.

Именно работа с листком бумаги помогает визуализировать ваше будущее приложение и поможет избежать проблем в будущем. Плюс к этому, у вас всегда перед глазами будет структура и это сэкономит время, т.к. ничего не нужно держать в голове.

Архитектура должна выглядеть примерно таким образом:

Типовая архитектура приложения



Проектирование ПО. Архитектура приложений

3

Рис. 7 – Типовая архитектура приложения

Она представляет из себя наброски, без конкретики в реализации. Помогает сориентироваться в проекте и выявить ключевые компоненты приложения. А также на этапе прототипирования поможет вам разделить приложение на независимые компоненты.

Но это еще не все. Можно пойти дальше и продумать внутренности каждого компонента. И тут уже помогут блок-схемы. Мы можем продумать логику каждого компонента, какие в нем будут функции, как информация будет перетекать из

одной функции в другую и в целом как поток данных будет гулять по вашему приложению.

Блок-схема выглядит таким образом:

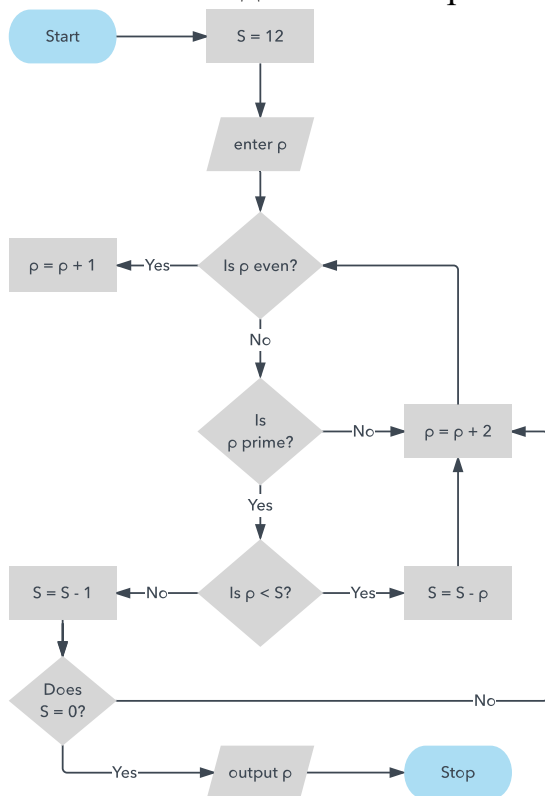


Рис. 8 – Блок схема

Здесь показана реализация одной функции. Все визуально просто и понятно. Подробнее про составление блок-схем можно почитать тут: <https://www.lucidchart.com/pages/ru/блок-схема>

Заключение

Теперь вы знаете не только маршрут по изучению Python, но и как его пройти быстро, правильно и максимально продуктивно.

Избегайте искушений бросить изучение, боритесь с факторами, которые мешают изучению языка.

Следуйте маршруту, используйте методики, которые описаны в книге. Не забывайте отдыхать и наслаждаться жизнью. Ну и самое главное, кайфуйте! Кайфуйте от программирования, от процесса изучения программирования.

```
print(“Ведь теперь вы программист.”)
```

Приложения

В данном разделе будут чек-листы по разделам и полезные ссылки, которые были по ходу данной книги.

Мои соц. сети

1. Telegram

канал:

<https://t.me/+ay4zBlMO7S9lOTZi>

2. YouTube:

<https://www.youtube.com/@kurushkin>

Книги к ознакомлению

1. «Простой Python» 2016 (автор Билл Любанович)

2. «Структуры данных в Python: начальный курс» 2022

Скачать Python

<https://www.python.org/>

Скачать IDE

1. Visual Code: <https://code.visualstudio.com/>

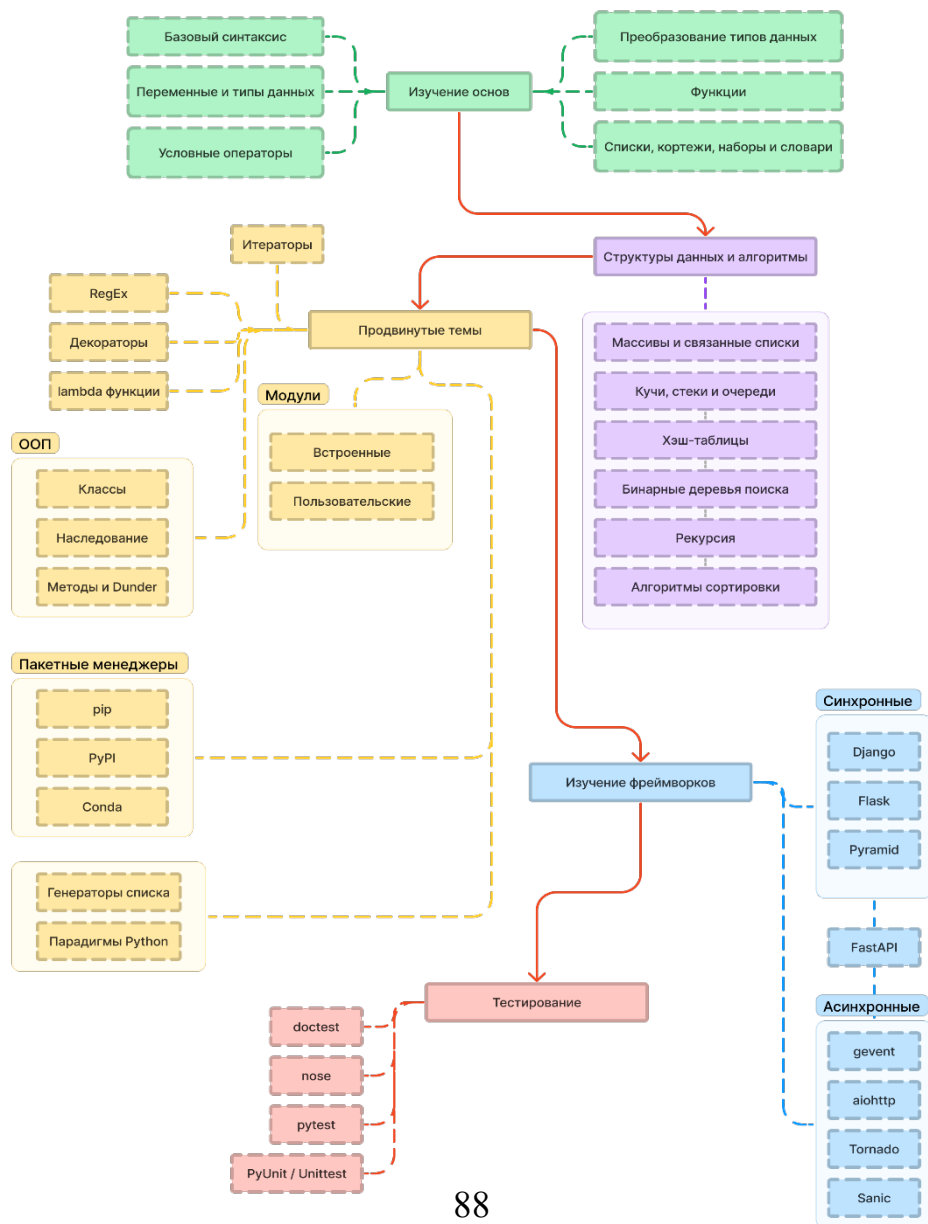
2. PyCharm:

<https://www.jetbrains.com/pycharm/>

Где решать задачи

<https://leetcode.com>

Дорожная карта по изучению Python



Порядок изучения Python

Чек-лист по главе книги

- ☐ Подготовка
 - ☐ Выбрать комфортное место
 - ☐ Подготовить блокнот
 - ☐ Выбрать напиток
 - ☐ Подготовить сладости
 - ☐ Придумать свой атрибут
 - ☐ Определить свою регулярность обучения (например, 2 часа в день)
 - ☐ Выбрать цель. Проект, программу, которую хотите разработать на Python
- ☐ Обучение
 - ☐ Пройти курс на YouTube
 - ☐ Прочитать книгу "Простой Python"
 - ☐ Решать задачи leetcode
 - ☐ Новая книга
 - ☐ Платный курс повышения квалификации (необяз.)
- ☐ Практика. Разработка собственного проекта
- ☐ Дополнительные навыки
 - ☐ Научиться печатать вслепую
 - ☐ Рисовать алгоритмы

Как изучать Python

Ежедневный чек-лист по изучению

Как смотреть видеокурс

- ☐ Посмотреть урок на скорости 1,5 без практики. Вдумчиво с пониманием, что делает лектор. Если не понятно - пересмотреть еще раз
- ☐ Записать новую или непонятную информацию из видео в блокнот
- ☐ Посмотреть урок на стандартной скорости. Протестировать код из видео
 - ☐ Напечатать код самостоятельно
 - ☐ Сделать комментарии к коду

Как читать книгу

- ☐ Прочитать главу
- ☐ Выписать новую или непонятную информацию в блокнот
- ☐ Протестировать код главы
- ☐ Написать комментарии к коду

Как решать задачи

- ☐ Решить 1 задачу уровня легко
- ☐ Решить 1 задачу уровня средне
- ☐ Решить 1 задачу уровня сложно